ONLINE BOOKSTORE

Proposed logical architecture derived from repository requirements

REQUIREMENTS REVIEW

No application code in reviewed snapshot

Buyer / Guest
Browse and manage cart; log in to buy

Seller
Own listings and order fulfilment

Admin
Accounts, listings and review moderation

Presentation layer | Responsive browser interface
Buyer storefront / Seller dashboard / Admin console
Framework not selected; gateway-hosted card entry is a proposed integration choice

HTTPS / TLS 1.2+

Card details to gateway over TLS

Never stored by bookstore (SEC-21)

Server request handling and security controls
Session validation | Role and ownership checks | Input validation | CSRF protection | Rate limits
Password hashing and admin MFA | Safe output encoding | Generic errors | Security audit events

Application layer | Logical modules

Identity and accounts
Registration, login, reset, sessions
Admin second factor | FR-1, 14

Checkout and orders
Server totals, coupons, stock protection
Verified payment -> order | FR-5, 11

Payment integration
Initiate, verify, reconcile; refunds
Persist reference and outcome | UC9

Catalogue and inventory
Search, details, seller-owned listings
Stock, approval visibility | FR-2, 3, 7

Ratings and reviews
Delivered purchase; content filter
Held reviews / ratings | FR-10

Invoices and notifications
Itemised receipts; email / SMS / OTP
Retry failed work | FR-6, 13

Cart and wishlist
Seller-specific items | FR-4, 15

Admin and moderation
Approve / suspend / remove | FR-12

Fulfilment and tracking
Own lines, status history | FR-8, 9

EXTERNAL SYSTEMS

TLS API

Payment gateway
Direct card entry; authorise / capture
Verified result and refund operations

TLS API

Email / notification provider
Order and status messages
Reset OTP / link; admin second factor

TLS API

Shipping / logistics provider
Tracking API + inbound status webhook

Shared data access | Parameterised SQL and transaction boundaries
Atomic order / stock / coupon updates; ownership-scoped queries; preserve order history

Background retry processor*
Durable SQL jobs; safe repeated delivery
Invoice generation / notification retries
Uses application modules + data access
No separate broker product assumed

SQL database | Product not selected
Accounts, roles, shared sessions, reset tokens | Books, seller listings, inventory, reservations*
Carts, wishlists, coupons | Orders and seller-specific lines, payment attempts*, status history
Reviews, invoice records, durable retry work* | Encrypted personal data; salted password hashes

Operational controls
Protected audit logs; encrypted backups
Secrets outside source; restricted access
Monitoring, health checks and restore tests*
Privacy correction / deletion workflow
Respect legal order / invoice retention

DESIGN AND DEPLOYMENT NOTES

* Proposed design additions: reservations, payment attempts, durable jobs and retry processing.
Scale application instances with shared sessions and coordinated SQL updates; routing / hosting remain to be selected.
Module grouping is proposed. All module persistence uses shared data access. Arrows show communication, not execution order.

Reviewed main @ 6bffd7fbf569 | 15 Oct 2026 | Sources: SRS, Security Requirements, Use Case Flow, Use Case Diagram, Test Plan